

Squally Line — *speaking script*

What each person says • in order Target **13:00** • allowed 8–15 min 7 speakers

0:00

PROJECT MANAGER

1:15

The client and the problem

01

"Hello, we're a seven-person team from KNUST, and this is **Squally Line** — a web platform we built for a real client, **[CLIENT NAME]**, a fashion designer based in **[CITY]**, Ghana."

"His business has three centres: bespoke tailoring; ready-to-wear sales; and in-studio consultations and fittings. Before Squally Line all of it ran on paper and WhatsApp — measurements in notebooks, quotes in chat, production tracked in his head. Customers couldn't browse the work, save measurements, or follow an order."

"Our brief was to find a real client, understand the problem, and solve it using software engineering — following the process. Over the next thirteen minutes we'll show the working system and the engineering path we took."

1:15

PROJECT MANAGER

1:30

How we gathered requirements

02

"We followed the standard life cycle: problem definition, requirements gathering and analysis, planning and design, implementation, testing, deployment, and now maintenance."

"There are many elicitation techniques; we chose the ones that fit a single expert client. Our **primary method was semi-structured stakeholder interviews** — open-ended questions about how he runs his studio. Our **second was direct observation, or contextual inquiry** — we watched him measure a client and run an order end to end. Those were supported by **document analysis** of his paper measurement books and WhatsApp chats, and by **iterative prototyping** of mock-ups reviewed with him."

"That gave us our requirements. The **functional** ones — what the system does — became features like measurement profiles, a catalogue, ordering, and a quote-and-approval workflow. The **non-functional** ones we grouped as the course teaches: security, performance and scalability, availability and reliability, usability, and compliance."

"We worked **Agile and incrementally**, prioritising with **MoSCoW**, shipping a working MVP first and updating the client weekly."

2:45

1:30

03

BACKEND LEAD

Architecture, decision records & the live URL

"Squally Line is a classic **three-tier client-server web application**. The front end is a **React single-page app** talking to a separate back end over a **REST API**. The back end is **Django with the REST Framework**, storing data in **PostgreSQL**. Both halves deploy independently on **Railway** — and you can use it live right now at the address on screen."

"Authentication is **stateless JWT** — short-lived access tokens with rotating refresh tokens. That was a deliberate decision, because it lets us scale the API horizontally later without sticky sessions."

"We logged decisions like that as **Architecture Decision Records** — ADRs: splitting the SPA from the API, JWT over sessions, and choosing **Paystack** because it's built for the Ghanaian market and settles in cedis."

"Our **third-party systems** are Paystack for payments and Railway for hosting, and every library we depend on is **permissively licensed** — React is MIT, Django is BSD. No copyleft in our tree, so we can legally hand this over as a private commercial product. We verified that with a licence checker."

4:15

1:45

04

FRONTEND LEAD + FRONTEND DEV 1 (DRIVES SCREEN)

Demo — the commerce spine

FRONTEND LEAD

"Here's the system in the customer's hands. The front door is our **Atelier** landing — a cinematic brand intro — and from there the customer enters the shop."

"Everything is live data. The catalogue is a **category tree** with search and filters. We separate **ready-to-wear products**, which have stock and buy directly, from **made-to-order styles**, which come next."

"Watch the buying flow. The cart is **owned by the server**, so it survives a refresh or a new device. At checkout the front end creates the order, then hands off to **Paystack** — we never touch card details. Paystack returns the customer, we verify, and the order appears in their history. This is the core path, working end to end."

FRONTEND DEV 1 (WHILE CLICKING)

"I'm paying with a Paystack **test card** so you see the full round trip — initialise, redirect, verify, confirm — with no real charge."

6:00

1:30

05

UI DESIGNER + FRONTEND DEV 1 (DRIVES SCREEN)

Demo — measurements, 3D & made-to-measure

UI DESIGNER

"This is where the platform earns its place in a tailoring business. Under **Measurements**, a customer keeps profiles — for themselves and the people they order for, like family — and every set is **versioned**, so history is kept, not overwritten."

"We wanted measurements to feel approachable, so we render a **real 3D figure** that responds as you fill it in, with an SVG fallback for weaker devices. The whole interface uses one design system — a **white, black and gold** palette matching the brand, with a dark and light theme. That consistency is a deliberate **usability** requirement, not decoration."

"To request a custom garment the customer fills this guided form — recipient, measurements, fabric, urgency, a consultation slot. Submitting **does not charge them**; it sends a request to the studio. Payment comes later, once the tailor sends a formal quote — exactly how real bespoke works, and it's what we show next."

7:30

1:30

06

FRONTEND DEV 2 + BACKEND DEV 1

Demo — the proposal & agreement workflow

FRONTEND DEV 2 (CUSTOMER SIDE)

"This is the heart of the system — the negotiation between customer and studio, done properly. On the **Agreement** tab the customer sees the studio's **proposal**: the garment, reference images, total price, deposit, completion date. Proposals are **versioned** — ask for changes and the studio issues a new version, with the whole exchange recorded."

"When the customer is happy, they hit **Accept** — and watch: the system turns that agreement into a real **order**."

BACKEND DEV 1 (SERVER SIDE)

"That one click does a lot on the server, all in one **atomic transaction**. Accepting a proposal automatically **creates the order, the order item with the full spec snapshotted in, and the payment schedule** — deposit and balance, each with a due date — and links it back to the proposal. We also built a **payment gate**: until the deposit is paid, the workshop details stay hidden. The rule 'no work starts without a deposit' is enforced in code, not on trust."

9:00

1:30

07

BACKEND DEV 2

Demo — the staff & admin roles

"The system has **three roles**, each seeing a different world. A **customer** sees the shop, their measurements and orders. An **apprentice** — tailoring staff — runs day-to-day shop-floor operations. An **admin** — the owner — does everything, including pricing and analytics."

"From the catalogue manager the studio adds products and styles and uploads images — and on upload the back end auto-converts every image to optimised **WebP** to stay fast. Under Consultations staff review requests, and in the **Proposal Editor** the studio composes the quote — the other end of the agreement the customer just accepted."

"The **Production Queue** tracks bespoke work through real stages — pattern-making, cutting, sewing, fitting, finishing, quality check — **sorted by deadline and flagging at-risk orders**. And the **Analytics** dashboard gives the owner revenue, top pieces, production stats and customer retention. That's the role-based control the brief asked for."

10:30

0:45

08

BACKEND DEV 1

Security & non-functional requirements

"Because this handles money and personal data, we treated the non-functional requirements as first-class. On **security**: JWT with rotating, blacklistable refresh tokens; every admin action protected by **role-based permissions** on the server, not just hidden in the UI; a strong password policy; and every Paystack webhook **signature-verified** before we trust it. On **performance** we paginate lists, optimise images to WebP, and refresh tokens silently. On **reliability**, checkout and proposal-acceptance run in **atomic transactions**, and the webhook is **idempotent**, so a repeated callback can't double-charge. Those qualities sit behind the features you just saw."

11:15

0:50

09

UI DESIGNER

Limitations & problems we faced

"No honest demo skips its limits. **What isn't done yet:** the system doesn't send email or SMS — configured but not wired. Refunds are modelled but have no full workflow. Some copy, like care and return policies, is still static. And a couple of interface pieces, like the top profile menu, are on placeholder data — which is why we've used the sidebar throughout."

"**The problems we faced** were as much practice as code. Coordinating **seven people on one codebase** meant living in Git branches and real merge conflicts. Learning **three.js** for the 3D figure took time. **Deploying to Railway** taught us hard lessons about environment config, CORS and static files. And our biggest management challenge was **scope creep** — which we controlled with MoSCoW and a scope freeze on the MVP."

12:05

0:45

10

BACKEND LEAD THEN PROJECT MANAGER

What comes next — is it scalable?

BACKEND LEAD

"Is it scalable? Yes — and we can be specific, because the signal to scale is real demand, not ambition. Asked 'what breaks first at ten thousand users?', we'd **harden the production config first** — secrets and debug into environment variables. Then it's textbook: **optimise** with indexes and caching, **scale vertically** on a bigger instance, then **scale horizontally** — and because our JWT auth is already stateless, many API instances run behind a load balancer with no redesign. First to **decompose**: media onto object storage and a CDN, plus a background worker for notifications."

PROJECT MANAGER

"On the roadmap after that: real email, SMS and WhatsApp notifications, a full refund flow, and eventually a mobile app. Commercially, since all seven of us are joint authors under Ghana's Copyright Act, we'll **assign the IP to the client on final payment**, hand over a signed MVP acceptance, and continue on a small hosting-and-support retainer."

12:50

0:30

11

PROJECT MANAGER

Close

"That's Squally Line — a real problem, for a real client, solved with a real engineering process from requirements to deployment. Our thanks to **[CLIENT NAME]** for trusting us with his studio. Try the live platform yourself at squallyline-frontend-production.up.railway.app. Thank you for watching."